Ageless Park Billing

Architecture, Working and Critical Review

Laravel 12 / MySQL monolith: park ticket billing, QR points wallets, ESP32 ride-reader redemption and mobile staff attendance CRM.

Reviewed	Source tree at C:\xampp\htdocs\Ageless_Park_Billing (version control: none found)
Date	10 Oct 2026
Method	Static reading of routes, domain, services, controllers, middleware, migrations, models, config, log and the firmware core loop. Tests were NOT executed.
Not reviewed	Dashboard, Ride, Package, RideSetup and WorkSite controllers; FirmwareBundle; Blade views; public JS; Python/browser tests; PHPUnit config

Verdict. The money and points core is carefully engineered: integer paise, idempotency keys, consistent lock ordering and a fail-closed pulse protocol. The surrounding access model, operations posture and code layering are well below that standard, and several policy rules (points expiry, attendance enablement, correction approval) are unsafe or undefined.

1. System architecture

A single Laravel 12 application (PHP 8.2+, MySQL via XAMPP) serves three clients with three different trust levels. There is no queue worker, no cache server and no separate API service: everything is synchronous in one process.

```
Cashier / Admin browser --> routes/web.php --> auth + active --+-- billing --> POS, sales, passes
                                                               +-- admin   --> rides, machines, users,
                                                                               settings, audit, CRM
Staff phone (browser GPS) -> /staff/*  ------> auth + active ---------------> AttendanceService
ESP32 QR readers (x25) ----> routes/api.php -> throttle + machine (bearer) --> RedemptionService

Controllers --> Services (Sale, Redemption, Attendance, Correction)
            --> Domain (pure rules: Money, RedemptionPolicy, AttendancePolicy)
            --> Eloquent models --> MySQL (InnoDB row locks for concurrency)
```

1.1 Layers and components

Layer	Files	Responsibility
Routing	routes/web.php, routes/api.php	Web (session auth) and machine API (bearer token). Middleware aliases: active, billing, admin, machine.
Middleware	ActiveUser, BillingAccess, AdminOnly, MachineAuth, SecurityHeaders	ActiveUser logs out disabled users and sessions whose password changed (password_stamp). MachineAuth hashes the bearer token (SHA-256) and looks up the machine, refreshing last_seen_at at most every 20 s. SecurityHeaders adds nosniff, frame-deny, no-store and a geolocation=(self) policy.
Controllers	15 controllers	HTTP validation and view rendering. Some also contain business logic (see Findings).
Services	SaleService, RedemptionService, AttendanceService, AttendanceCorrectionService, Audit, Qr, FirmwareBundle, SqliteToMysqlImporter	Transactional business operations with idempotency and locking.
Domain	Money, RedemptionPolicy, AttendancePolicy	Framework-free rules: paise arithmetic and tax rounding, redemption denial precedence, haversine geofence and overlap maths.
Models	User, ParkPass, Package, Ride, Machine, Sale, SaleItem, PointEntry, Redemption, AuditLog, Setting, WorkSite, AttendanceSession / Punch / Correction	Eloquent, mostly guarded=['id']. The UtcDateTime cast stores attendance times explicitly in UTC.
Console	ParkInstall, AdminPassword, DatabaseCheck, ImportSqlite	Install, admin password reset, DB diagnostics, one-time SQLite to MySQL import.
Firmware	firmware/AgelessCameraReader, AgelessParkReader	ESP32-CAM QR reader (quirc decoder), pending-transaction persistence in NVS, relay pulse output.
Ops tooling	01-04 .bat files, setup-linux.sh, tools/	Windows setup and start, DB login check, admin password reset.

1.2 Data model

Table	Key columns / constraints	Purpose
users	role (admin/cashier/staff), active, employee_code (unique), trade AND designation, work_site_id, attendance_enabled	Logins and staff profile. trade and designation duplicate each other.
packages / rides	price_paise, points, validity_hours / points_per_play, pulse_ms, cooldown_seconds, soft deletes	Sellable tickets and ride pricing.
machines	serial (unique), ride_id, token_hash (unique, sha256), enabled, last_seen_at	One reader per ride; token shown once.

Table	Key columns / constraints	Purpose
park_passes	credential 'QR-' + 32 hex (unique), balance, status, expires_at	The wallet. ONE balance and ONE expiry per pass.
sales / sale_items	request_id (uuid unique), fingerprint, totals in paise, receipt_settings JSON snapshot, void fields	Bills, idempotent by request_id.
point_entries	delta, balance_after, kind (issue / topup / redeem / ride_refund / adjustment / void)	Append-only points ledger.
redemptions	unique(machine_id, request_id), credential_hash, status, code, points, authorized_at	Each ride scan and its lifecycle.
work_sites	lat/lng, radius_m, max_accuracy_m, enforce_geofence, timezone	Attendance locations.
attendance_sessions	open_user_id (unique, NULL when closed), UTC started_at/ended_at, site_snapshot JSON, needs_review	One shift per row.
attendance_punches	request_id unique, unique(session, type), lat/lng/accuracy, distance, location_status, ip, user_agent	Raw in/out evidence, never overwritten.
attendance_corrections	status, before_values / after_values JSON, reviewed_by	Correction requests and approvals.
settings / audit_logs	single JSON row / action, entity, details JSON	Park configuration and audit trail.

2. How each module works

2.1 Authentication and roles

- Login normalises the email (LOWER/TRIM lookup, tolerant of imported mixed-case accounts), then Auth::attempt requires active=true. Throttle: 5 attempts per minute per email+IP. The session ID is regenerated and a password_stamp stored.
- Every request passes ActiveUser: a deactivated user or a changed password kills the session at once. Role is read from the database per request, so demotion is immediate.
- Roles: admin (everything), cashier (POS, sales, passes via the 'billing' middleware), staff (staff portal only). Staff are redirected to /staff; the '/' route is a closure that picks the dashboard or staff home.

2.2 Selling tickets (POS)

- The cashier builds a cart; the page carries a UUID request_id. SaleController validates (max 30 distinct lines, qty 1-99, money regex, payment cash/upi/card).
- SaleService fingerprints [user, payload]. In one transaction it first checks for an existing sale with that request_id: same fingerprint returns the same bill (replay), a different fingerprint is rejected. A unique-key race falls back to the replay path.
- Pricing: reads active packages, sums subtotal and points, takes the MINIMUM validity_hours across lines. Discount is admin-only and capped at the subtotal. Tax is computed in integer basis points with exact half-up rounding; 'inclusive' keeps the total, 'exclusive' adds tax. Cash received must cover the total.
- Pass: either a new pass is created (random 128-bit credential) or an existing credential is locked FOR UPDATE and topped up (expiry = later of existing and new). Balance cap is 1,000,000,000 points.
- Writes: sale (receipt settings snapshotted), invoice number PREFIX-yymmdd-id, sale_items, pass balance/expiry, a PointEntry, an audit row. The bill page renders the QR as server-side SVG.
- Void (admin): allowed only for an untouched first-sale pass (one sale, one ledger entry, no redemptions, balance unchanged). Balance goes to 0, the pass is blocked and a negative ledger entry is written. Anything else needs 'separate reconciliation'.

2.3 Ride redemption protocol (ESP32 to server)

```
Reader                                    Server (RedemptionService)
------                                    --------------------------
scan QR, persist pending in NVS
POST /api/machine/redeem {credential, request_id}
                                          lock machine row; replay check on (machine, request_id)
                                          lock pass row; RedemptionPolicy::deny() in this order:
                                            machine disabled > ride disabled > machine pending >
                                            unknown QR > pass blocked > expired > cooldown > balance
                                          denied  -> row status=denied (422)
                                          allowed -> DEDUCT points, ledger 'redeem',
                                                     status=approved, 30 s window (200)
one relay pulse (pulse_ms)
POST /machine/transactions/{id}/ack       approved -> completed ; 'uncertain' -> operator review
{outcome: pulsed}
clear NVS pending
(lost reply: retry the SAME request_id; server replays the stored result)
```

- Points are debited BEFORE the pulse (fail-closed for the park's revenue). After 30 s an un-acked approval is flipped to 'uncertain' (REVIEW_REQUIRED), including by a plain GET status call.
- While any approved/uncertain transaction exists the reader answers MACHINE_PENDING. An admin resolves it (confirm = keep the debit, refund = return points) only after disabling the reader and waiting out the 30 s window; the action is audited.
- Other machine endpoints: config (ride, pulse, cooldown), check (balance/expiry), status by request_id. Rate limits: 3600/min per IP and 120/min per token.

2.4 Passes, adjustments, machines, settings

- Admins can block/unblock a pass and adjust points with a mandatory reason (ledger kind 'adjustment', audited). Already-issued approvals are not remotely cancelled.
- Machines: create (token shown once), update (ride change blocked while a transaction is pending), rotate token (also blocked while pending), download the Config.h firmware config after re-proving the token.
- Settings: one JSON row (park name, colours, invoice prefix, tax rate/mode, receipt width/footer, logo). Each sale keeps its own snapshot.

2.5 Staff attendance

- Punch-in/out (POST /staff/punch, 20 per minute): the browser supplies GPS and a UUID request_id; the SERVER supplies identity, site and time (UTC). The user row is locked; a unique open_user_id guarantees one open shift.
- AttendancePolicy computes haversine distance against the site config snapshotted on the session. Status: recorded / inside / outside / poor_accuracy / not_configured. Punch-in is refused when the result needs review; punch-out is always allowed but flagged. Shifts over 24 h are flagged.
- Corrections: staff request a fix with a reason (one pending per date). An admin approves with new local times (end after start, max 36 h, not in the future, no overlap) or rejects. Original punches stay untouched; before/after snapshots and an audit row are written.
- Admin CRM: filter by date (max 366 days), site, user, trade, status, search; summary counters; audited CSV export with UTF-8 BOM. Staff see today's status, history and requests, and can change their password (min 12 characters).

2.6 Users, audit and tooling

- Admins manage users (no delete). Guards: you cannot disable or demote yourself; the last active admin is protected by a locked count.
- Audit log: Audit::record() runs inside the same transaction as the change. Logged: sales, voids, pass changes, machine operations, settings, user changes, attendance events, exports.
- Importer: ImportSqlite / SqliteToMysqlImporter migrates the earlier SQLite deployment to MySQL; the .bat files set up and start the app.

3. What is done well

- **Money is integers.** Paise end to end, regex-validated input, exact rounding, upper limits. No float accounting.
- **Idempotency where it matters:** sales (request_id + fingerprint), redemptions (machine + request_id), punches (request_id + fingerprint), each with conflict detection.
- **Consistent lock ordering and retries** (machine then pass; user then session), with database unique constraints as a second line of defence (open_user_id, unique(session, type)).
- **Append-only evidence:** restrictOnDelete on all foreign keys, no delete routes for users, immutable punches, receipt and site snapshots, before/after correction records.
- **Fail-closed hardware protocol:** debit first, NVS-persisted pending state, never replay a pulse, operator reconciliation for uncertainty.
- **Security basics:** hashed machine tokens shown once, CSRF via Laravel, security headers, CSV formula-injection guard on the sales export, session invalidation on password change.
- **Breadth of tests:** about 100 feature test methods plus domain smoke tests and a firmware integration test.

4. Critical findings

Severity reflects likely impact on money, data integrity or security in a live park. File references are relative to the project root.

#	Sev	Finding	Evidence / why it matters
1	HIGH	A GET request silently enables attendance for everyone	StaffPortalController::home (L13-19) assigns a default site and forces attendance_enabled=true for any active user who opens /staff. This overrides an administrator who deliberately disabled attendance, making the flag decorative; a GET should not write state.
2	HIGH	Admins can approve their own corrections	AttendanceCorrectionService::review never checks reviewer != requester. Admins can use the staff portal, and approval rewrites session times. No separation of duties on the payroll-relevant path.
3	HIGH	Points expiry rules are unsound or undefined	SaleService L22, L33-37: one pool, one expiry. Mixed-validity carts expire at the shortest validity (a 30-day ticket plus a 2-hour add-on expires in 2 h). Topping up an expired pass revives old leftover points. Refunds add points but do not extend an expired pass.
4	HIGH	Production posture	APP_ENV=local, DB user root, http URL, SESSION_SECURE_COOKIE=false, file sessions/cache/rate-limit store, and the log shows php artisan serve (the dev server) handling requests while 25 readers poll. The log also shows earlier DB credential/driver confusion.
5	HIGH	No version control, no backups	The directory is not a git repository (releases are zips plus SHA256SUMS). No backup/restore tooling found. For a system holding money and attendance evidence this is the largest operational risk.
6	MED	Weak and inconsistent credential policy	Admin-created users need only 8 characters; the self-service change needs 12 with mixed case and digits. Login throttle is per email+IP only, so credential stuffing across many emails from one IP is unlimited.
7	MED	One lost acknowledgement halts a ride	MACHINE_PENDING blocks the reader until an admin disables it and waits 30 s (RedemptionService L54-68). Safe, but operationally heavy across 25 rides; there is no timed auto-resolution policy.
8	MED	QR secret exposure and API surface	GET /machine/check accepts the credential in the query string (logged by web servers); legacy /qr/* routes and a qr_code alias add surface. A printed QR is a bearer secret: anyone who photographs it can spend it. Reader traffic may be plain HTTP on the LAN (firmware option). Firmware HTTPS certificate validation was not verified.

#	Sev	Finding	Evidence / why it matters
9	MED	Attendance totals silently exclude flagged shifts	AttendanceAdminController L27-29 drops needs_review sessions from the headline total and sums rows in PHP rather than SQL. No auto-close for forgotten punch-outs, so the next day's punch-in is blocked until the person punches out, creating a flagged ~20 h shift. The review threshold (24 h) differs from the correction cap (36 h).
10	MED	Attendance proof is advisory	GPS is self-reported; geofence is off by default; no device binding or photo, so buddy-punching (using a colleague's login) is not technically prevented. Poor accuracy blocks punch-in even when the geofence is off (AttendancePolicy L25).
11	MED	Missing retail controls	No discount reason; one aggregate tax figure (no per-line or GST breakup); no cashier shift/day close; only first sales can be voided; sales CSV export not audited; no check that park_passes.balance equals the sum of point_entries.delta.
12	LOW	Inconsistent layering	Sales and redemption logic is in services, but correction requests, pass adjust/status, last-admin guards and CSV streaming live in controllers. No Form Requests, no policies/gates, no enums: role strings and status literals ('approved', 'uncertain', ...) are repeated across many files.
13	LOW	Dense, hard-to-read code	Controller methods of 400+ characters on one line. Money::paise() is reused to convert the tax rate to basis points (SaleService L29), which works by naming accident.
14	LOW	Schema and migration debt	trade and designation duplicated; an empty migration (2026_10_09_160013_add_role_to_users_table); a data seed inside a schema migration; mixed time conventions (attendance explicit UTC, created_at/expires_at in app timezone); the '/' closure route breaks route:cache.
15	LOW	Concurrency not proven by tests	lockForUpdate is a no-op on SQLite. If PHPUnit runs on SQLite, the locking design is untested. phpunit.xml was not read.
16	LOW	Privacy / retention	Staff coordinates, IP, user agent and customer phone numbers are stored indefinitely with no retention policy.

5. Recommended actions

Priority	Action
1 (this week)	Initialise git and push to a private remote. Add automated daily MySQL dumps stored off the machine and perform a test restore.
1	Remove the state write from StaffPortalController::home; add a reviewer != requester rule to corrections; unify password rules (12+) and add a per-IP login throttle.
1	Decide and document the points-expiry rules (per-lot expiry or a fixed policy), then enforce them in SaleService and the refund path.
2 (this month)	Run under Apache/Nginx + PHP-FPM with a dedicated least-privilege DB user, APP_ENV=production, HTTPS and secure cookies; move sessions, cache and rate limits to the database or Redis.
2	Remove GET /machine/check and the /qr/* aliases; require HTTPS for readers; verify certificate validation in the firmware; consider QR rotation or short-lived pass tokens.
2	Add a ledger reconciliation command (balance vs ledger), a cashier day-close report, discount reasons and per-line tax; audit the sales export.
3 (next quarter)	Introduce enums, Form Requests and policies; move controller logic into services; drop the duplicate trade/designation column; remove the empty migration; normalise timestamps to UTC.
3	Run the concurrency tests against MySQL; add auto-close for stale open shifts and a defined retention policy for location data.

5.1 Limits of this review

This is a static review. Tests were not run, runtime behaviour was not observed, and the unreviewed files listed on the cover may contain further issues, or already-correct handling of points above (notably the dashboard's data exposure to cashiers, the firmware's TLS handling and the Blade views' output escaping). Findings 8 (firmware TLS) and 15 (test database) are explicitly unverified.